Proceedings of the 7th International Conference on Innovative Data Communication Technologies and Application (ICIDCA-2025)
IEEE Xplore Part Number: CFP25CR5-ART; ISBN: 979-8-3315-1414-3

2025 7th International Conference on Innovative Data Communication Technologies and Application (ICIDCA) | 979-8-3315-1414-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICIDCA66325.2025.11280486

Generative AI in Agile Software Development: A Comprehensive Survey

Nagarjun V. Malladi
Research Scholar, School of Engineering
Anurag University
Hyderabad, India
venkatanagarjunmalladi@gmail.com

Sudheer Reddy, K.
School of Engineering
Anurag University
Hyderabad, India
sudheercse@gmail.com

***Abstract*—Generative artificial intelligence (GenAI) adoption is reforming the software development life cycle (SDLC) practices, and Agile is no exception to this trend. The use of generative AI tools, huge language models, and domain-specific copilots is on the rise in automating coding, refactoring, testing, documentation, and requirements engineering tasks. Although initial data suggest benefits in productivity, quality, and time-to-market, GenAI also poses challenges related to technical dependability, integration, ethical challenges, and organizational maturity. However, despite this rapid evolution, currently lack systematic insight into how generative AI aligns with Agile values and practices, resulting in a fragmented research terrain. This survey critically reviews the studies related to generative AI in Agile-based SDLC. The contribution is a structured overview of applied ML across SDLC stages, Agile practices, and types of generative models through an easily comprehensible taxonomy. Second, by investigating such studies in terms of research methods, application areas, evaluation measures, and tools, it further synthesizes methodological findings. Faster code generation, automated testing and documentation, better prioritization of the backlog, higher developer productivity, and cost savings are contrasted against hallucination, scalability, bias, and adoption challenges. The survey also highlights opportunities and future research scope, including human–AI collaboration, benchmark standards, interpretable and trustworthy AI, domain-specific frameworks, and others. This paper advances the understanding of the implications of generative AI for Agile SDLC, from both the opportunities and challenges perspectives, and lays the foundational ground for future research and practice.**

***Index Terms*—Generative AI, Agile Software Development, SDLC, Large Language Models, Explainable AI**

I. Introduction

The SDLC has been in use for decades as a foundational model for organizing software engineering activities, encompassing requirements engineering, design, implementation, testing, deployment, and maintenance. Agile approaches have emerged over the past few decades as the primary paradigm for organizing SDLC-related activities, and they are characterized by adaptability, iterative delivery, stakeholder collaboration, and continuous improvement. While helping organizations adapt to evolving needs and market dynamics at a faster pace, it has also introduced higher levels of complexity, faster modes of delivery, and the challenge of maintaining quality uniformity through distributed teams. At the same time, progress in artificial intelligence (AI) has opened new avenues for enhancing, automating, and augmenting software engineering. Among these advances, generative AI is a disruptive technology that could be a revolutionary for many Agile-based SDLC practices.

Generative AI, based on large language models and other generative architectures, can generate a variety of software artifacts, including code, test cases, documentation, and design alternatives. These tools appear to speed up development workflows, while also reducing manual effort and freeing up creative space for teams [25], [29], [36]. However, this is preliminary research, and the results are based on industry pilots. In Agile, on the other hand, various types of copilots are increasingly being adopted for pair programming, requirement refinement, and automated testing [41], [47]. However, while there is plenty of promise here, leveraging generative AI within Agile also creates new challenges. Meanwhile, difficulties like hallucination in code generation, infusion into Agile ceremonies, bias in generated outputs, and accountability as to who will be responsible for bugs in AI-generated code increase the complexity of deploying to the masses.

These developments highlight the need to conduct a systematic survey of the current status of research and practice on generative AI in Agile methodologies and the SDLC. In summary, this paper fills the gap by compiling existing studies in a structured taxonomy, using them to evaluate trends in methods, perceived advantages and obstacles, and exploring future perspectives. This survey contributes to the existing literature in three ways. First, it provides an overview of existing literature by classifying it based on SDLC stages, Agile practices, and generative model types. Second, it extracts methodological knowledge by analysing research methods, areas of application, evaluation measures, and tools used. Third, it suggests future research paths primarily focused on human–AI collaboration, standard standards, explanations, domain-specific frameworks, and sustainability, in addition to some proposed advantages and disadvantages.

The paper is organized as - Section 2 presents literature review, synthesizing prior work into thematic areas. Section 3 introduces a taxonomy of GenAI applications in Agile SDLC. Section 4 presents methodological insights, while Section 5 highlights the benefits and opportunities that these insights offer. Section 6 concludes with reflections.

Stock market trading has become more and more popular worldwide, and many individuals now include it into their daily routines in an attempt to make money [1]. However, it is difficult to anticipate because of the intricacy of stock market data. Predicting certain future events by examining historical data is known as forecasting [2]. It addresses business, economics, finance, and industry. However, the chance to consistently generate wealth through the stock market has improved as a result of developments in technology, which also assist professionals in determining the most important indicators for better forecasting. Time analysis is a common component of predicting difficulties [3-4]. Finding trends and patterns is much simpler when time-series data is analyzed. This facilitates the discovery of data phases or cycles, patterns, and trends [6]. Early awareness of a bullish or negative trend in the stock market helps investors make prudent financial decisions. Additionally, the patterns aid in identifying the top-performing businesses throughout a specific time frame [8-9].

II. Literature Survey

The rapid evolution of GenAI has introduced new possibilities across the SDLC, reshaping traditional practices and enabling novel forms of automation, collaboration, and decision support. While classical AI and machine learning approaches have long been applied to specific stages such as requirements engineering, defect prediction, or testing [7], [9], [20,21,22], the emergence of generative models marks a paradigm shift by directly producing artifacts including code, test cases, documentation, and design prototypes [25], [14], [29]. In parallel, Agile methodologies continue to dominate contemporary software engineering, emphasizing iterative delivery, customer collaboration, and responsiveness to change [2], [5-6]. The intersection of these domains—Generative AI and Agile practices—has sparked increasing academic and industrial interest, reflected in case studies, systematic reviews, and pilot implementations [27], [28], [35], [36]. However, few concerns relating to explainability, ethical use, data privacy, and integration challenges remain underexplored [37], [40], [50], [65]. To capture this evolving terrain, the following review synthesizes existing research into thematic areas that illuminate the role of GenAI in Agile-based SDLCs.

A. Generative AI in SDLC

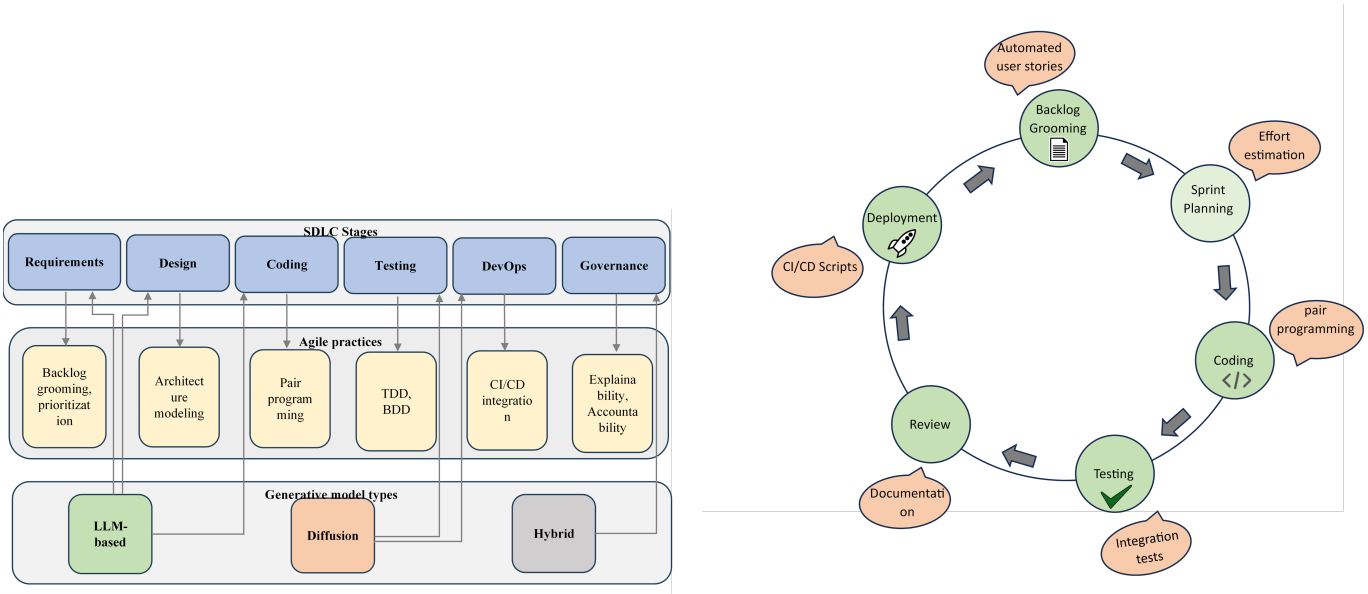
This section focuses on integrating generative AI into various phases of the SDLC, including requirements, design, coding, testing, deployment, and maintenance. SHAFIQ et al. [9] examine the use of machine learning in the phases of software development, examining how it interacts with tools and methods to determine whether ML is advantageous for particular phases or techniques, emphasizing recent findings. Authors [16-17] propose a novel method for resolving inconsistent software models by generating specific, grouped repair options. After being tested on ten case studies, it was shown to be workable, scalable, and consistent with developer choices. Jaakko Sauvola et al. [13] discuss the impact of generative AI on software development, along with four potential future scenarios, productivity improvements, ethical concerns, and recommendations for additional research. Authors of [26] presented the HACAF model, which is tested using a combination of techniques, for the adoption of generative AI in software engineering. According to the findings, adoption is driven by workflow compatibility. It proposes more longitudinal research for verification. Takura Mbizo et al. [25] examined developers' opinions of GenAI technologies in software projects, emphasizing the advantages of workflow and potential ethical concerns. Results from tests on a variety of experts suggest potential benefits, and caution about potential long-term difficulties.

Rasmus Ulfsnes et al. [26] conducted interviews to investigate the impact of GenAI on software workflows, finding that it enhances learning, creativity, and productivity. Reduced team collaboration is a disadvantage; work automation is a benefit. Future research on the broader effects should be conducted. Asha Rajbhoj et al. [27] tested a methodical prompting strategy for ChatGPT use across SDLC stages on a advanced application. Although it increases productivity, it has non-deterministic issues that need to be validated by SMEs. Tajinder Kumar et al. [34] examined how generative AI affects software development, with a particular emphasis on automation in requirements analysis, design, coding, and testing. Faster creativity, lower errors, and more production are highlighted, albeit there may be obstacles to complete integration. Authors [40] examine the explainability of GenAI in software engineering in this work.

Authors [41] compared artifacts produced by AI and manual methods to investigate the integration of LLMs into software testing education. Authors of [38] introduced a multi-dimensional evaluation methodology while assessing many generative AI models for Python code development. Moritz Mock et al. [42] compared fully automated and collaborative patterns while introducing a Generative AI-assisted Test-Driven Development (TDD) methodology. The findings indicated that while GenAI can increase productivity, it needs oversight, particularly from non-experts. Mario Simaremare et al. [42] examine the integration of GenAI with agile product development in software startups in this article. It accelerates development and enhances the quality of the final product by identifying use cases, validating them through expert workshops. Authors [30-32] introduced AI-Analyst, an AI-assisted framework for SDLC that uses a transformer model to optimize resource allocation, cut expenses by 53.33%, and enhance decision-making with high accuracy and performance.

B. Generative AI within Agile Methodologies

This section covers research on how AI/GenAI tools support Agile practices (Scrum, Kanban, BDD, requirement prioritization, sprint planning, retrospectives). Biesialska et al. [2] examined 88 cases and connected Agile Software Development (ASD) and Big Data Analytics (BDA). It highlights how BDA can enhance efficiency throughout the ASD lifespan, with a particular focus on analytics in key strategic areas. Chatterjee et al. [4] examine the factors that influence the adoption of



(a) Taxonomy of Generative AI in Agile SDLC

(b) Workflow of GenAI Integration into Agile Sprint Cycle

Fig. 1: (a) Taxonomy of Generative AI in Agile SDLC, and (b) Workflow of GenAI Integration into Agile Sprint Cycle.

AI-integrated CRM systems (AICS) by agile firms. It presents valuable insights and suggestions for future study, following the identification of the primary adoption reasons through a survey and PLS-SEM. LUNESU et al. [5] propose a method for modeling risk variables in agile development, utilizing Monte Carlo simulations and software process simulation modeling (SPSM). The results demonstrate its efficacy in risk assessment. Henry Edison et al. [6] propose that the purpose of the paper is to help practitioners choose the best large-scale agile methodologies by comparing them (SAFe, LeSS, Scrum-at-Scale, DAD, Spotify). It highlights difficulties, elements that contribute to success, and areas that require more investigation. The Souza et al. [8] technique, which combines Scrum, ontologies, and BDD to address challenges in LMS development, is introduced in the paper as ScrumOntoBDD. It reduces ambiguities through better planning, analysis, and communication, but it doesn't provide strong BDD assistance. CHORAS et al. [10] study on the use of software development metrics in Agile projects in SMEs is presented in this paper. It provides a list of verified metrics that can be adjusted and utilized to enhance project outcomes.

Katarzyna Biesialska et al. [28] examine 88 papers and evaluate Agile Software Development (ASD) using Big Data Analytics (BDA). It highlights important areas of concentration, points out the paucity of empirical research, and urges more extensive, industry-based future research. XIAOYAN ZHAO et al. [62] assessed five data-generating strategies to enhance Agile cost estimation with sparse project data. WGAN demonstrated the best overall performance, highlighting the benefits of synthetic data. Kari Sainio et al. [49] presented Prompt Engineering, which uses ChatGPT to solve problems in Agile Project Management. It highlights ChatGPT's shortcomings in crucial jobs while showcasing its assistance with task automation and suggesting best practices for integrating AI. TASNEEM et al. [74] examined traditional, semi-automated, and AI-based approaches to requirement prioritizing in Agile development. It enhances the efficacy of Agile by highlighting the advantages, disadvantages, challenges, and unmet research needs of the reprioritization process. Shafir et al. [75] proposed cultural context and customer engagement as critical components of successful Agile development in Bangladesh. It emphasizes the need for localized Agile methods.

C. Productivity, Collaboration, and Case Studies of Generative AI in Agile SDLC

This section focuses on empirical evidence and case studies highlighting productivity improvements, developer experience, and team dynamics. Takura Mbizo et al. [25] examined developers' opinions of generative AI technologies in software projects, emphasizing the advantages of workflow and potential ethical concerns. Results from tests on a variety of experts suggest potential benefits, but also caution about potential long-term difficulties. Rasmus Ulfsnes et al. [26] conducted interviews to investigate the impact of GenAI on software workflows, finding that it enhances learning, creativity, and productivity. Reduced team collaboration is a disadvantage; work automation is a benefit. Asha Rajbhoj et al. [27] tested a methodical prompting strategy for ChatGPT use across SDLC stages on a advanced application. Rasmus Ulfsnes et al. [31] present an empirical investigation of the effects of GenAI tools, such as ChatGPT, on software development in this work. The findings impact collaboration and learning loops while also demonstrating efficiency, quicker learning, and changed teamwork dynamics. Mariana Coutinho et al. [36] examine the incorporation of generative AI technologies into software

development in this study. Results indicate productivity improvements, but also highlight drawbacks.

Tajinder Kumar et al. [34] examined how GenAI affects software development, with a particular emphasis on automation in requirements analysis, design, coding, and testing. Faster creativity, lower errors, and more production are highlighted, albeit there may be obstacles to complete integration. Ramazan Yilmaz et al. [35] examined the impact of ChatGPT on programming instruction and found that students' motivation, self-efficacy, and computational thinking improved. Authors [41] compared artifacts produced by AI and manual methods to investigate the integration of LLMs into the training in software testing. According to the results, LLMs improve learning and speed, but cannot replace manual testing; future research will examine a broader range of tools. Jensen et al. [70] examined how software development companies manage expectations for AI tools, emphasizing modifications made upon assessment. It highlights the importance of managing expectations to successfully implement AI, revealing both unique and common expectations among enterprises. Adzgauskaite et al. [73] use TOE framework to examine the factors that contribute to the success of remote teams in the the development oft of Agile software.

D. Explainability, Trust, and Ethics in Generative AI–Enabled Software Engineering

This section explores explainability of code generated by AI, responsible AI in SDLC, trust in hybrid human–AI teams, and AI ethics in Agile contexts. Caldwell et al. [11] present an Agile-based framework for human-AI collaboration that has been evaluated in test settings, such as video games. Although it aims to enhance performance and decision-making, the real-world transferability of this approach poses challenges. Ismaeel Al Ridhawi et al. [19] suggested a Plug and Play AI framework that allows autonomous ML tuning and selection for edge devices in smart cities and the Internet of Things. Lameck Mbangula Amugongo et al. [33] propose an ethical paradigm for AI in healthcare in this paper, with a focus on "ethics by design." With a case study on AI-enabled mHealth applications, it incorporates ethical concepts throughout development and provides practical solutions. Authors [40] examine the explainability of GenAI in software engineering in this work. To enhance human-centered technical development, it offers four XAI features for GenAI that were developed during engineering workshops. David Lo [45] examined AI for Software Engineering (AI4SE), outlining obstacles and suggesting solutions to develop reliable and cooperative AI. It sees AI4SE as a key component in the advancement of Software Engineering 2.0. Authors [51-52] combined knowledge from industry interviews and literature to suggest six fundamental steps for creating explainable systems.

Authors [50] used interviews to examine explainability in industrial AI and uncovered changing stakeholder needs throughout the AI lifecycle. It offers insights into sociotechnical design. Crockett et al. [48] investigated the obstacles to SME adoption of ethical AI by comparing 77 toolkits to 33 standards. It shows little use of the toolbox and little helpful advice. Barbosa et al. [56] and [57] addressed ethics in tech design by introducing an epistemic tool that extends semiotic engineering. While it lacks factual validation, it facilitates moral thought. Laato et al. [65] explored integrating AI governance into SDLC by conducting expert interviews to identify 20 governance themes. It highlights the complexity of governance at all levels. Qinghua Lu et al. [67] paves the way for responsible AI by addressing system-level architecture, development processes, and governance. It aims to bridge the gap between ethical concepts and real-world implementation, although it has not been tested for practical use. Shahriar et al. [60] examined privacy concerns in AI systems at various stages of their life cycle. It examines privacy-enhancing alternatives, obstacles, and current laws, emphasizing concerns such as non-compliance, identification, and incorrect decisions.

E. Challenges, Risks, and Limitations of Generative AI in SDLC and Agile Frameworks

This section sheds light on adoption barriers, risks related to quality, accountability, legal compliance, scalability, and long-term organizational impacts. AbuSalim et al. [7] emphasize qualities such as completeness and testability. The article examines techniques for evaluating the quality of Software Requirements Specifications (SRS) and their impact on the software development lifecycle and decision-making. Geismann et al. [12] emphasize both the cyber and physical layers; the study examines model-driven security techniques for cyber-physical systems (CPS). It highlights the shortcomings and unmet research needs of seven CPS-specific strategies. Akdur et al. [13] present MAPforES, an embedded software characterisation model used in consumer electronics, automotive, and defense. By identifying patterns, the results demonstrate enhanced organizational and project behavior. Authors [15] use data from Mozilla Firefox to assess IR and DL methods for traceability between bug reports and test cases. Despite its poor overall performance, LSI performed better than the others and showed promise for semi-automatic use. Authors [12-17] suggest incorporating a versioning system into parametric design processes to enhance analytics, feedback, comparison, and version management.

Alaca et al. [18] present a framework for assessing the quality and usability of MAS DSMLs in the paper "AgentDSM-Eval." It provided information on developer uptake, domain coverage, and performance indicators when applied to a well-known DSML. Md. Shamsujjoha et al. [19] Mobile app developers face challenges due to platform dependencies. Using a thorough literature analysis, this paper reviews Model-Driven Development (MDD) methodologies, highlighting their benefits, limitations, and possible future directions. Jin Zhang and Jingyue Li [22] examined 83 works on neural network-based safety-critical system testing and verification, categorizing approaches, identifying problems, and highlighting areas that require further investigation, such as interpretability and robustness. NIJAT RAJABLI et al. [23] explored software verification and validation methodologies for autonomous cars,

looking at challenges, safety rules, current strategies, and future research needs to ensure secure autonomy and dependable AI. Gilda Taranto-Vera [18] examined the development of data mining, emphasizing important algorithms and programs such as WEKA and RapidMiner, as well as the expanding role of machine learning in information extraction from vast data.

Adrien Berthelot et al. [33] used Stable Diffusion as a case study to present an LCA-based technique for assessing the environmental impact of generative AI services. It highlights the high cost of AI models in terms of resources and energy. Takura Mbizo et al. [25] explored how generative AI tools impact software development, highlighting advantages such as increased productivity and automation, while also raising concerns about long-term dependency, job security, and ethical implications. Mousa Al-kfairy [45] examined the use of generative AI in large and small businesses, emphasizing both the difficulties and the efficiency improvements. It offers future research directions for digital transformation, as well as a roadmap for overcoming obstacles. VALERIO TERRAGNI et al. [46] explored the transition of programming from machine code to libraries and APIs, emphasizing how greater abstraction levels have streamlined software development for complex systems and improved developer efficiency.

Authors [41] examined AI applications at various phases of PLM and provided a well-organized research roadmap. It highlights the possibilities, challenges, and existing implementation gaps of AI in smart manufacturing. R. Beckers et al. [48] provided guidance for medical physicists on applying the EU MDR to AI-based medical devices, outlining safety assurance, quality systems, and regulatory procedures to promote clinical adoption and enhance patient outcomes. Sakao et al. [49] proposed a data lake-based LCE method utilizing AI and IoT, which has been validated in industrial environments. It makes lifetime modifications more precise and quicker. Integration and scalability are key limitations and potential areas for improvement. Daniyan et al. [51] present a predictive maintenance PLMS system with AI integration in the research and evaluate it using railcar bearing data. It produced precise RUL forecasts. Benefits include improved monitoring; drawbacks include scalability and data quality issues. Authors [58][61][64], inspired by software lifecycles, present a transparency approach for dataset development. Conceptually tested, it enhances documentation and responsibility. Adoption complexity is one of the limitations. Authors [53-54] examined DevOps focus on quality, classifying contributions according to lifecycle stages. It exposes integration issues, identifies gaps, and provides recommendations for further study in AI-driven DevOps.

Johnson et al. [55] highlighted talent shortages by analyzing BD&AI workforce gaps using job data and bibliometrics. It provides a roadmap for aligning industry demands with academic instruction. Lichun et al. [63] examined AI applications in petroleum exploration, with a focus on the use of computer vision, deep learning, and machine learning. Sartori et al. [58] suggested an integrated EIA approach for building design that combines GBRS and LCA. It enhances transparency and emphasizes stage-based applicability, but it requires stronger 3D integration. Future research will concentrate on tool development. SOFIAN et al. [60] present a systematic mapping analysis on AI approaches in software engineering in this work. It finds patterns and gaps in SE phases. PEIZHENG LI et al. [62] presented RLOps, an intelligent network management system that combines Open RAN and reinforcement learning (RL). It lists difficulties, suggests best practices for developing automated models, and showcases a platform for data analytics. Ghoroghi et al. [69] investigated the combination of Life Cycle Assessment and machine learning to produce more precise environmental impact analyses.

Authors [70] examined LCA applications in buildings and pointed out gaps and current research trends. Although it offers suggestions for future research, it lacks thorough case studies and insights into real-world applications. Authors [59] assessed how student projects are affected by sequential, iterative, and ”hands-off” development methodologies. Findings from more than 100 students reveal variations in output, collaboration, and product quality, which inform the design of software courses. Santhosh S et al.,[67] proposed a Parameter-Ranking Priority Level Weightage algorithm that streamlines the software development decision-making process by comparing and evaluating cloud service providers based on key criteria. Wolf et al. [68] examined LCA databases and technologies that support the EU's Level(s) framework for evaluating the environmental impacts of buildings and suggested evaluation standards to improve sustainability and meet EU policy objectives. Stefan Walter [75] proposed an AI-powered approach to supply chain monitoring that enhances performance and flexibility globally. It integrates cloud computing, AI, and data analytics, resulting in improved planning and production outcomes. Barakabitze et al., [72] examine QoE management in 5G/6G networks in their study, along with a roadmap for multimedia services that utilizes self-driving 3D networks, ML-based controllers, and softwarization.

Overall, the literature indicates that GenAI has the potential to significantly enhance Agile software engineering by streamlining repetitive tasks, accelerating prototyping, and augmenting decision-making during iterative cycles. At the same time, empirical studies suggest that improvements in productivity and collaboration are accompanied by emerging risks related to model transparency, accountability, and workforce adaptation. While early case studies demonstrate promising results in domains such as code generation, testing automation, and requirements engineering the findings also highlight the importance of balancing efficiency gains with responsible deployment practices and organizational readiness. The reviewed studies collectively underscore the dual imperative of leveraging GenAI to strengthen Agile development while addressing the ethical, methodological, and socio-technical challenges that could shape its long-term impact on the SDLC. Building on these insights, the subsequent sections delve into thematic contributions in detail and identify research directions.

III. TAXONOMY OF GENAI IN AGILE SDLC

The integration of GenAI into Agile software engineering can be understood through a taxonomy defined by three dimensions: the stage of the SDLC, the Agile practice supported, and the type of generative model employed. Across the SDLC, generative AI has been applied in requirements and planning, design, implementation, testing, deployment, and governance. In requirements engineering, models such as large language models assist in user-story drafting, backlog grooming, and reprioritization, directly improving estimation and planning accuracy [29], [38], [74]. For design and architecture, emerging generative approaches facilitate the exploration of alternatives and automate parametric design documentation [18]. The most extensive applications appear in implementation, where copilots such as GitHub Copilot and ChatGPT accelerate coding, refactoring, and bug repair, and are increasingly validated in empirical studies [25], [28], [44]. Testing and quality assurance represent another active domain, with generative AI applied to automated unit test generation, TDD/BDD integration, and defect localization [47], [41]. In DevOps and deployment, hybrid methods integrating reinforcement learning and data synthesis are used for pipeline authoring and quality-aware CI/CD practices. Governance, explainability, and ethics are integral to all stages, where explainable AI (XAI), accountability frameworks, and privacy-aware adoption strategies are being increasingly emphasized [65-66]. To illustrate the distribution of studies across these dimensions, representative examples are summarized in Table I, highlights key SDLC stages, Agile practices, generative model types, and contributions, providing a concise view of how the literature aligns with the proposed taxonomy. Analysis of this mapping highlights several clear trends. Large language models have emerged as the dominant technology for generative support in coding and collaborative review, with growing evidence of productivity improvements [25], [29]. Testing is transitioning from simple unit-level automation to full integration with TDD and BDD practices, with encouraging results in both industrial and educational contexts [47], [41][43]. Requirements engineering is another expanding area, where generative AI supports user-story creation and dynamic backlog reprioritization [80]. At the same time, governance, explainability, and privacy are becoming integral to the adoption of generative AI in Agile, reflecting a growing awareness of ethical and organizational challenges [65], [72]. Overall, the taxonomy reveals a progression from early tool demonstrations toward broader organizational adoption, with hybrid approaches combining LLMs, data synthesis, and policy layers playing a crucial role in ensuring responsible deployment of generative AI in Agile SDLC.

Figure 1a illustrates the taxonomy of generative AI in Agile SDLC, mapping SDLC stages to corresponding Agile practices and generative model types. It highlights how requirements, coding, testing, DevOps, and governance activities are supported by LLMs, hybrid systems, and emerging generative design approaches, offering a structured overview of generative AI integration across development processes.

IV. METHODOLOGICAL INSIGHTS

A review of the literature reveals that diverse research methodologies have been employed to study the role of generative AI in Agile software engineering. Case studies dominate the field, particularly those examining how teams use tools like ChatGPT and GitHub Copilot in real-world software projects, highlighting productivity benefits and collaboration challenges [29], [24], [41]. Systematic literature reviews also contribute by synthesizing findings across multiple studies, particularly in areas such as requirement reprioritization [80], DevOps practices [59], and testing verification [22], providing structured insights into emerging patterns. Empirical surveys and pilot experiments are increasingly common, focusing on developer perceptions, organizational readiness, and productivity effects of GenAI adoption [27], [28], [70]. This methodological diversity reflects the novelty of the field, where researchers balance exploratory case-driven evidence with broader comparative reviews to capture a rapidly evolving terrain.

Generative AI in Agile SDLC has been applied across a wide range of domains. In healthcare, case studies demonstrate their role in supporting mobile health applications and ensuring governance of AI-enabled software [37]. In the financial sector, cost estimation and optimization studies highlight the relevance of hybrid models for resource management and project planning [36]. Education is another emerging domain, with studies focusing on the use of Copilot and ChatGPT for testing education and improving computational thinking [11], [39]. Software startups offer fertile ground for experimentation, where generative AI helps accelerate design and coding under resource constraints [48]. Broader enterprise-level applications emphasize productivity gains, improved collaboration, and alignment with organizational strategies, reflecting growing interest in scaling AI adoption across large and distributed Agile teams [25], [35], [10].

The comparative mapping in Table II Highlights the methodological diversity in current research on generative AI for Agile SDLC. Case studies and pilot projects predominate, reflecting the experimental and practice-driven stage of this field, while systematic reviews and comparative analyses provide a structured understanding of emerging patterns. The breadth of application domains, spanning healthcare, finance, education, startups, and large enterprises, demonstrates the adaptability of generative AI across both specialized and general contexts. Evaluation metrics remain multidimensional, balancing technical outputs, such as code quality and test coverage, with human-centered outcomes, including developer satisfaction and organizational efficiency. The concentration of studies around widely available tools such as ChatGPT and GitHub Copilot indicates the prominence of general-purpose LLMs. Still, the growing presence of hybrid and domain-specific copilots suggests an evolving ecosystem customized to address specific challenges. Collectively, these methodological insights underscore the need for both rigorous empirical studies and context-aware evaluations to more effectively capture the delicate role of generative AI within Agile frameworks.

TABLE I: Representative Studies Categorized by SDLC Stage, Agile Practice, Generative Model Type, and Contribution

Ref	SDLC Stage	Agile Practice	Gen Model Type	Contribution
[25]	Coding	Pair programming	LLM	Vision of GenAI copilots in future SE
[29]	Coding, Requirements	Review, backlog grooming	LLM	Case study on ChatGPT accelerating Agile workflows
[47]	Testing	TDD	LLM	Preliminary results of GenAI for automated test generation
[41]	Testing	Agile testing	LLM	Case study on Copilot and ChatGPT in testing education
[80]	Requirements	Backlog prioritization	Hybrid	Systematic review of Agile requirement reprioritization
[65]	Governance	Agile governance	Hybrid	Insights on responsible ML and AI governance
[72]	Governance	Privacy risk management	Hybrid	Survey on AI lifecycle privacy risks

TABLE II: Summary of Methodologies, Domains, Metrics, and Tools in Generative AI Studies for Agile SDLC

Ref	Methodology	Application Domain	Evaluation Metrics	Tools/Platforms
[29]	Case study	Agile enterprise project	Productivity, time efficiency	ChatGPT
[36]	Pilot case study	Software teams (general)	Developer satisfaction, productivity	GitHub Copilot
[41]	Case study (education)	Software testing education	Testing quality, learning outcomes	ChatGPT, Copilot
[74]	Systematic literature review	Agile requirement engineering	N/A (review synthesis)	N/A
[42]	Empirical analysis	Agile cost estimation	Cost efficiency, estimation accuracy	Hybrid AI + data synthesis
[47]	Experimental study	Agile testing (TDD)	Code quality, test coverage	LLM prototypes
[37]	Case study	Healthcare (m-health apps)	Governance, compliance, accountability	Domain-specific AI copilots
[44]	Comparative study	Coding (Python)	Code quality, correctness	LLMs (Python code models)
[71]	Framework evaluation	Enterprise/finance	Cost–benefit analysis	Hybrid SDLC analysis framework
[46]	Multi-case study	Software enterprises	Developer perception, adoption barriers	Mixed (Copilot, ChatGPT, etc.)

Evaluation of GenAI in Agile contexts relies on a set of recurring metrics. Productivity is the most frequently reported metric, measured in terms of reduced development effort, faster task completion, and accelerated project timelines [29], [38]. Code quality is another critical dimension, assessed through defect rates, maintainability, and accuracy of generated or repaired code [44], [16]. Developer satisfaction and user experience are emphasized in case studies, often highlighting how generative AI influences motivation, cognitive load, and perceived usefulness [36], [70]. Cost and time efficiency appear in studies that analyze resource optimization and project planning, where hybrid models combining data synthesis with AI assistance demonstrate tangible benefits [42], [71]. Together, these metrics offer a multifaceted view of the impact of generative AI, capturing both technical outputs and human-centered outcomes.

A variety of tools and platforms underpin studies reviewed. ChatGPT and GitHub Copilot are most widely investigated, particularly in coding and testing contexts [29], [41], [44]. Bard and other large language model interfaces are explored less extensively, but are recognized for their potential in assisting with requirement engineering and documentation tasks. Domain-specific copilots are also emerging, offering customized support for specialized contexts such as healthcare software, DevOps pipelines, and Agile project management [37], [42]. The insights suggest that the tool ecosystem is expanding rapidly, with general-purpose LLMs providing broad support while niche copilots evolve to address domain-specific challenges. This variety reflects an ongoing transition from experimental use of mainstream platforms to the integration of specialized GenAI systems into Agile SDLC processes.

Figure 1b illustrates the workflow of integrating GenAI into the Agile sprint cycle, demonstrating how AI tools support backlog grooming, sprint planning, coding, testing, reviews, and deployment. It emphasizes the continuous feedback loop of Agile while highlighting AI-driven contributions such as automated user stories, code generation, test creation, documentation, and CI/CD pipeline support.

V. BENEFITS AND OPPORTUNITIES

Generative AI is increasingly recognized for its ability to accelerate software development activities, particularly in coding and refactoring tasks. Large language models, such as ChatGPT and GitHub Copilot, assist developers in producing functional code snippets, automating repetitive tasks, and identifying opportunities for refactoring that enhance maintainability and readability. Comparative studies demonstrate that generative models can achieve significant reductions in coding effort while improving repair accuracy, making them valuable assets in Agile sprints where quick turnaround is essential [25], [29], [44]. These capabilities enable teams to allocate more time to design considerations and high-level problem-solving rather than routine programming. Another significant contribution is the automation of testing and documentation. GenAI has been applied to produce unit and integration tests, create acceptance criteria in behavior-driven development, and assist with preparatory testing artifacts in educational settings. Studies show that AI-generated tests not only reduce the manual workload but also improve coverage and consistency across iterative Agile cycles [41], [47]. Documentation tasks, often neglected in fast-paced Agile environments, are supported through automated generation of comments, design notes, and user guides, thereby enhancing communication within distributed teams [16], [40]. These advances reduce technical debt and improve the sustainability of Agile projects. Table III highlights key benefits of GenAI in Agile SDLC, mapping

them to Agile activities. GenAI also supports requirement prioritization and backlog grooming, two critical activities for Agile planning and responsiveness. Automated techniques based on hybrid models and data synthesis approaches provide more consistent backlog reprioritization and effort estimation, reducing human bias and improving alignment with evolving customer needs [42], [74]. By generating user stories, acceptance criteria, and scenario descriptions, GenAI contributes to more structured sprint planning, allowing product owners and developers to make informed trade-offs during iterative releases. Beyond this, GenAI enhances developer productivity and creativity. Studies on developer perceptions report that generative copilots not only save time but also act as cognitive aids, suggesting new approaches or coding strategies that developers may not have initially considered [36], [70]. The collaborative use of GenAI is further associated with improved team dynamics, where hybrid human–AI partnerships support pair programming, retrospectives, and knowledge sharing [35].

Ultimately, the combined benefits result in potential cost reductions and accelerated time-to-market. By reducing manual effort in coding, testing, and documentation, organizations can shorten development cycles and optimize resource allocation. Empirical analyses of cost estimation frameworks integrating GenAI highlight measurable improvements in efficiency and project predictability.

Despite its apparent benefits, the adoption of GenAI in Agile SDLC is accompanied by a series of technical, process-related, ethical, and organizational challenges. From a technical perspective, one of the most persistent concerns is hallucination, where large language models generate plausible but incorrect or insecure code snippets. This affects correctness and reliability, making AI-generated artifacts risky to integrate directly into production systems. Scalability is also a significant limitation, as the performance of generative models can degrade when dealing with large, complex projects or enterprise-scale codebases, raising questions about consistency and maintainability.

Process challenges also hinder smooth integration into Agile methodologies. Agile ceremonies, such as sprint planning, backlog grooming, and retrospectives, are highly collaborative; however, current generative tools are primarily designed for individual developer assistance [29], [49]. Version control systems encounter complications when AI tools generate code at scale, often resulting in merge conflicts or redundant commits that disrupt workflows [70]. In CI/CD pipelines, the introduction of AI-generated artifacts can complicate automated builds and testing, especially if traceability and reproducibility are not maintained [59], [68]. These challenges reflect a gap between the rapid pace of AI-driven automation and the structured practices of Agile software engineering. Table IV outlines challenges in adopting GenAI for Agile SDLC.

Ethical concerns are equally significant, particularly in terms of transparency, bias, and accountability. Studies emphasize the opacity of GenAI systems, which makes it difficult for teams to understand how code or documentation was produced [40], [65]. Accountability is another open issue, as responsibility for errors or failures becomes blurred when outputs co-produced by humans and AI tools [50]. These ethical dilemmas demand the development of explainable AI and governance frameworks that align with Agile principles.

Organizational challenges encompass adoption barriers, skill gaps, and cultural resistance. Many teams remain cautious about integrating GenAI due to uncertainties about productivity gains versus risks. Developers may lack the necessary training to effectively prompt and validate AI outputs, resulting in inconsistent adoption and uneven benefits. Resistance also arises from cultural factors, as some Agile teams prioritize human collaboration and view AI as disruptive to established practices. At an enterprise level, aligning the integration of GenAI with organizational strategies requires careful planning, stakeholder buy-in and establishment of clear policies.

VI. CONCLUSIONS AND FUTURE SCOPE

The study examined the emerging role of GenAI in the context of the SDLC, based on Agile frameworks, and provided a structured taxonomy, methodological insights, and a balanced discussion of the benefits, challenges, and future directions. The review emphasized that GenAI is beginning to transform various aspects of SDLC, including requirements engineering, coding, testing, documentation, and governance. The case studies and empirical evaluations reveal improvements in productivity, efficiency, and collaboration with increasing opportunities for creativity and innovation in Agile teams. Simultaneously, the literature highlights significant threats, including hallucination in code generation, integration issues with Agile ceremonies and CI/CD pipelines, opacity and accountability challenges, as well as organizational impediments to adoption and cultural inertia. Moving forward, future research should focus on further investigating models of human–AI collaboration so that AI is integrated as more than a coding assistant, but as an active participant in Agile practices. Standardized standards will need to be developed to assess tools consistently across projects and contexts. Trustworthy AI and explainability remain essential to prevent the reckless use of AI, particularly in domains that can benefit from enhanced AI, such as healthcare, finance, and justice. This will further strengthen adoption within niche industries by implementing domain-specific frameworks that encompass regulatory and compliance obligations. Third, sustainability needs to be advocated as a mainstream guideline, with research in areas such as energy-efficient AI models and the environmental impacts associated with new AI-enabled development pipelines.

Although studies have proven productivity improvements with hit-or-miss copilots, much remains to be explained regarding how exactly AI can become a player in Agile ceremonies, such as sprint planning, retrospectives, and daily stand-ups. Another priority is to establish standardized standards for assessing GenAI within SDLC. Existing evaluations rely on adhoc metrics, making it impossible to compare performance across tools and settings.

TABLE III: Benefits of Generative AI in Agile SDLC with Supporting References

References	Benefit	Agile Activity Supported
[25], [29], [44]	Faster code generation and refactoring	Sprint implementation, pair programming, code review
[41], [47], [16], [40]	Automated testing and documentation	TDD/BDD workflows, integration testing, user story acceptance criteria, project documentation
[42], [74]	Improved requirement prioritization and backlog grooming	Backlog management, sprint planning, and effort estimation
[36], [70], [35], [12]	Enhanced developer productivity and creativity	Pair programming, retrospectives, prototyping, and team collaboration
[42], [71], [38]	Cost reduction and time-to-market acceleration	Agile planning, release mgt., enterprise level SDLC optimization

TABLE IV: Challenges and Open Issues in Generative AI for Agile SDLC

References	Category	Key Issues
[25], [44], [46]	Technical	Hallucination, code correctness, and scalability limitations
[29], [49], [59], [68], [70]	Process	Integration with Agile ceremonies, version control conflicts, and CI/CD disruptions
[40], [50], [65], [72]	Ethical	Lack of transparency, bias in outputs, and unclear accountability
[15], [36], [38], [27], [70]	Organizational	Adoption barriers, developer skill gaps, cultural resistance, and strategic misalignment

REFERENCES

[1] Hamidur Rahman, Ricky Stanley D'cruze, Mobyen Uddin Ahmed, Rickard Sohlberg, Tomohiko Sakao, and Peter Funk. (2022). Artificial intelligence-based life cycle engineering in industrial production: a systematic literature review. DOI:10.1109/ACCESS.2022.3230637

[2] Biesialska, K., Franch, X., & Muntés-Mulero, V. (2021). Big Data analytics in Agile software development: A systematic mapping study. Information and Software Technology. doi:10.1016/j.infsof.2020.106448

[3] Qian, B., Su, J., Wen, Z., Jha, D. N., Li, Y., (2020). Orchestrating the Development Lifecycle of Machine Learning-based IoT Applications. ACM Computing Surveys, 53(4), 1–47. doi:10.1145/3398020

[4] Chatterjee, S., Chaudhuri, R., Vrontis, D., Thrassou, A., & Ghosh, S. K. (2021). Adoption of artificial intelligence-integrated CRM systems in agile organizations in India. Technological Forecasting and Social Change, 168, 120783. doi:10.1016/j.techfore.2021.120783

[5] Maria Ilaria Lunesu, Roberto Tonelli, Lodovica Marchesi (2021). Assessing the Risk of Software Development in Agile Methodologies Using Simulation. IEEE. 9, DOI:10.1109/ACCESS.2021.3115941

[6] Edison, H., Wang, X. (2021). Comparing Methods for Large-Scale Agile Software Development: A Systematic Literature Review. IEEE Transactions on Software Engineering, 1–1. doi:10.1109/tse.2021.3069039

[7] Samah W. G. AbuSalim, Rosziati Ibrahim, (2020). Analyzing the impact of assessing requirements specifications on the software development life cycle. Springer., Doi:10.1007/978-3-030-58817-5_46

[8] Lopes de Souza, P., Lopes de Souza, W.(2021). ScrumOntoBDD: Agile software development based on scrum, ontologies and behaviour-driven development. Journal of the Brazilian Computer Society, 27(1). doi:10.1186/s13173-021-00114-w

[9] Saad Shafiq, Atif Mashkoor, Christoph Mayr-Dorn (2021). A Literature Review of Using Machine Learning in Software Development Life Cycle Stages. DOI:10.1109/ACCESS.2021.3119746

[10] Michał Choraś, Tomasz Springer, Rafał Kozik, Lidia López, Silverio Martínez-Fernández, Prabhat Ram, Pilar Rodríguez (2020). Measuring and improving agile processes in a small-size software development company. IEEE. 8, DOI:10.1109/ACCESS.2020.2990117

[11] Sabrina Caldwell, Penny Sweetser, Nicholas O'Donnell. (2022). An agile new research framework for hybrid human-AI teaming: Trust, transparency, and transferability. ACM. 12(3), doi:10.1145/3514257

[12] Geismann, J. (2020). A systematic literature review of model-driven security engineering for cyber-physical systems. Journal of Systems and Software, 110697. doi:10.1016/j.jss.2020.110697

[13] Akdur, D., Say, B., (2020). Modeling cultures of the embedded software industry: feedback from the field. Software and Systems Modeling, 20(2), 447–467. doi:10.1007/s10270-020-00810-9

[14] Gadelha, G., Ramalho, F., (2021). Traceability recovery between bug reports and test cases-a Mozilla Firefox case study. Automated Software Engineering, 28(2). doi:10.1007/s10515-021-00287-w

[15] Kretschmer, R., Khelladi, D. E., (2021). Transforming abstract to concrete repairs with a generative approach of repair values. Journal of Systems and Software, Doi:10.1016/j.jss.2020.110889

[16] Cristie, V., Joyce, S. C. (2021). Versioning for parametric design exploration process. Automation in Construction, 129, doi:10.1016/j.autcon.2021.103802

[17] Ridhawi, I. A., Otoum, S., Aloqaily, M., (2020). Generalizing AI: Challenges and Opportunities for Plug and Play AI Solutions. IEEE Network, 1–8. doi:10.1109/mnet.011.2000371

[18] Alaca, O. F., Tezel, B. T.(2021). AgentDSM-Eval: A framework for the evaluation of domain-specific modeling languages for multi-agent systems. Computer Standards & Interfaces, doi:10.1016/j.csi.2021.103513

[19] Shamsujjoha, M., Grundy, J., Li, L., Khalajzadeh, H. (2021). Developing Mobile Applications Via Model Driven Development: A Systematic Literature Review. Information and Software Technology, 140, 106693. doi:10.1016/j.infsof.2021.106693

[20] Zhang, J., Li, J. (2020). Testing and verification of neural-network-based safety-critical control software: A systematic literature review. Information and Software Technology, 106296. doi:10.1016/j.infsof.2020.106296

[21] Rajabli, N., Flammini, F., Nardone, R., (2021). Software Verification and Validation of Safe Autonomous Cars: A Systematic Literature Review. IEEE Access,. doi:10.1109/access.2020.3048047

[22] Taranto-Vera, G., Galindo-Villardón, P. (2021). Algorithms and software for data mining and machine learning: a critical comparative view from a systematic review of the literature. The Journal of Supercomputing

[23] Jaakko Sauvola, Sasu Tarkoma, Mika Klemettinen,(2024). Future of software development with generative AI.10.1007/s10515-024-00426-z

[24] Daniel Russo. (2024). Navigating the complexity of generative ai adoption in software engineering. ACM. 33(5), Doi:10.1145/3652154

[25] Takura Mbizo, Grant Oosterwyk (2024). Cautious Optimism: The Influence of Generative AI Tools in Software Development Projects. Springer.

[26] Rasmus Ulfsnes, Nils Brede Moe (2024). The role of generative ai in software development productivity: A pilot case study. Springer.

[27] Asha Rajbhoj, Akanksha Somase (2024). Accelerating Software Development Using Generative AI: ChatGPT Case Study. doi: 10.1145/3641399.3641403

[28] Biesialska, K., Franch, X., (2021). Big Data analytics in Agile software development: A systematic mapping study. Information and Software Technology, 132, doi:10.1016/j.infsof.2020.106448

[29] Adrien Berthelot, Eddy Caron, Mathilde Jay. (2024). Estimating the environmental impact of Generative-AI services using an LCA-based methodology. Elsevier. 122, doi: 10.1016/j.procir.2024.01.098

[30] S. R. Cheekati and K. Sudheer Reddy, "Machine Learning Strategies for Nifty50 Index Trading Optimization," 2025 International Conference on Advanced Computing Technologies (ICoACT), Sivalasi, India, 2025, pp. 01-06, doi: 10.1109/ICoACT63339.2025.11004954.

[31] Rasmus Ulfsnes, Nils Brede (2024). Transforming Software Development with Generative AI: Empirical Insights on Collaboration Workflow.

[32] Mariana Coutinho, Lorena Marques, Anderson Santos, Marcio Dahia (2024). The Role of Generative AI in Software Development Productivity: A Pilot Case Study. ACM, Doi:10.1145/3664646.3664773

[33] Lameck Mbangula Amugongo, Alexander Kriebitz, Auxane Boch, and Christoph Lütge. (2023). Operationalising AI ethics through the agile

software development lifecycle: a case study of AI-enabled mobile health applications. Springer, Doi:10.1007/s43681-023-00331-3

[34] Tajinder Kumar, Vishal Garg, Sachin Lalar, and Rajinder Kumar. (2023). Measuring Impact of Generative AI in Software Development and Innovation. Springer., pp.1-11. Doi:10.1007/978-981-97-1682-1_6

[35] Ramazan Yilmaz, Fatma Gizem Karaoglan Yilmaz. (2023). The effect of generative artificial intelligence (AI)-based tool use on students' computational thinking skills, programming self-efficacy and motivation. Elsevier. 4, pp.1-14. Doi: 10.1016/j.caeai.2023.100147

[36] Jiao Sun, Q. Vera Liao, Michael Muller, Mayank Agarwal, Stephanie Houde, Kartik Talamadupula, and Justin D. Weisz. (2022). Investigating explainability of generative AI for code through scenario-based design. ACM, pp.212 - 228. Doi: 10.1145/3490099.3511119

[37] Susmita Haldar, Mary Pierce, and Luiz Fernando Capretz. (2025). Exploring the Integration of Generative AI Tools in Software Testing Education: A Case Study on ChatGPT and Copilot for Preparatory Testing Artifacts in Postgraduate Learning. IEEE. 13, pp.46070 - 46090. DOI:10.1109/ACCESS.2025.3545882

[38] Xiaoyan Zhao, Zulkefli Mansor, Rozilawati Razali, Mohd Zakree (2025). Advancing Agile Software Cost Estimation through Data Synthesis: A Comparative Analysis of Five Generation Techniques. IEEE. 13, pp.63219 - 63236. DOI:10.1109/ACCESS.2025.3558715.

[39] Dominik Palla and Antonin Slaby. (2025). Evaluation of Generative AI Models in Python Code Generation: A Comparative Study. IEEE. 13, pp.65334 - 65347. DOI:10.1109/ACCESS.2025.3560244

[40] Mousa Al-kfairy. (2025). Strategic Integration of Generative AI in Organizational Settings: Applications, Challenges and Adoption Requirements. IEEE., pp.1-14. DOI:10.1109/EMR.2025.3534034

[41] Valerio Terragni, Annie Vella, Partha Roop (2025). The Future of AI-Driven Software Engineering. ACM., Doi:10.1145/3715003

[42] Moritz Mock, Jorge Melegati (2024). Generative AI for Test Driven Development: Preliminary Results.10.1007/978-3-031-72781-8_3

[43] Mario Simaremare, Triando, and Sergio Rico. (2024). Exploring the Potential of Generative AI: Use Cases in Software Startups. Springer., pp.3-11. Doi:10.1007/978-3-031-72781-8_1

[44] Kari Sainio, PekkaAbrahamsson(2023). Prompt patterns for agile software project managers: First results. Springer.

[45] David Lo. (2023). Trustworthy and synergistic artificial intelligence for software engineering: Vision and roadmaps. IEEE., pp.1-17. DOI:10.1109/ICSE-FoSE59343.2023.00010

[46] Larissa Chazette, Jil Klünder, Merve Balci. (2022). How can we develop explainable systems? insights from a literature review and an interview study. ACM., Doi:10.1145/3529320.3529321

[47] Wang, L., Liu, Z., Liu, A., (2021). Artificial intelligence in product lifecycle management. The Intl. Journal of Advanced Manufacturing Technology, 114 (3-4), doi:10.1007/s00170-021-06882-1

[48] Beckers, R., Kwade, Z. (2021). The EU medical device regulation: Implications for artificial intelligence-based medical device software in medical physics. Physica Medica, 83, doi:10.1016/j.ejmp.2021.02.011

[49] Sakao, T., Funk, P., Matschewsky, J., Bengtsson, M. (2021). AI-LCE: Adaptive and Intelligent Life Cycle Engineering by applying digitalization and AI methods – An emerging paradigm shift in Life Cycle Engineering. Procedia CIRP, doi:10.1016/j.procir.2021.01.153

[50] Dhanorkar, S., Wolf, C. T., Qian, K., Xu, A (2021). Who needs to know what, when?: Broadening the Explainable AI (XAI) Design Space by Looking at Explanations Across the AI Lifecycle. Designing Interactive Systems Conference 2021. doi:10.1145/3461778.3462131

[51] Daniyan, I., Muvunzi, R., & Mpofu, K. (2021). Artificial intelligence system for enhancing product's performance during its life cycle in a railcar industry. Procedia CIRP, 98, doi:10.1016/j.procir.2021.01.138

[52] Hutchinson, B., Smart, A., Hanna, A. (2021). Towards Accountability for Machine Learning Datasets. ACM Conference on Fairness, Accountability, and Transparency. doi:10.1145/3442188.3445918

[53] Alnafessah, A., Gias, A. U., Wang, R., Zhu, L.(2021). Quality-Aware DevOps Research: Where Do We Stand? IEEE Access, 9, doi:10.1109/access.2021.3064867

[54] Keeley Crockett, Edwin Colyer, Luciano Gerber, and Annabel Latham. (2023). Building trustworthy AI solutions: A case for practical solutions for small businesses. IEEE. 4(4),DOI:10.1109/TAI.2021.3137091

[55] Johnson, M., Jain, R., (2021). Impact of Big Data and Artificial Intelligence on Industry: Developing a Workforce Roadmap for a Data Driven Economy. Global Journal of Flexible Systems Management, 22(3), 197–217. doi:10.1007/s40171-021-00272-y

[56] Barbosa, S. D. J., Barbosa, G. D. J., (2021). A Semiotics-based epistemic tool to reason about ethical issues in digital technology design and development. Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency. doi:10.1145/3442188.3445900

[57] KUANG, L., LIU, H., REN, Y., LUO, K. (2021). Application and development trend of artificial intelligence in petroleum exploration and development. Petroleum Exploration and Development, 48(1), 1–14. doi:10.1016/s1876-3804(21)60001-0

[58] Sartori, T., Drogemuller, R., Omrani, S. (2021). A schematic framework for Life Cycle Assessment (LCA) and Green Building Rating System. Journal of Building Engineering, 38, doi:10.1016/j.jobe.2021.102180

[59] Samuli Laato, Teemu Birkstedt, Matti Mäntymäki (2022). AI governance in the system development life cycle: Insights on responsible machine learning engineering. ACM. doi:10.1145/3522664.3528598

[60] Hazrina Sofian, Nur Arzilawati Md Yunus, and Rodina Ahmad. (2022). Systematic mapping: Artificial intelligence techniques in software engineering. IEEE. 10. DOI:10.1109/ACCESS.2022.3174115

[61] Qinghua Lu, Liming Zhu, Xiwei Xu, (2022). Towards a roadmap on software engineering for responsible AI. ACM., Doi:10.1145/3522664.3528607

[62] Peizheng Li, Jonathan Thomas, Xiaoyang Wang, Ahmed Khalil, (2022). Rlops: Development life-cycle of reinforcement learning aided open ran. IEEE. 10, DOI:10.1109/ACCESS.2022.3217511

[63] Ali Ghoroghi, Yacine Rezgui, Ioan Petri. (2022). Advances in application of machine learning to life cycle assessment: a literature review. Springer. 27, doi:10.1007/s11367-022-02030-3

[64] Abdulrahman Fnais, Yacine Rezgui, Ioan Petri, Thomas Beach, Jonathan Yeung, (2022). The application of life cycle assessment in buildings: challenges, and directions for future research. Springer. 27. Doi:10.1007/s11367-022-02058-5

[65] Rafal Włodarski, Aneta Poniszewska-Marańda (2022). Impact of software development processes on the outcomes of student computing projects: A tale of two universities. Elsevier.

[66] Sakib Shahriar, Sonal Allana, Seyed Mehdi Hazratifard (2023). A survey of privacy risks and mitigation strategies in the artificial intelligence life cycle. IEEE. DOI:10.1109/ACCESS.2023.3287195

[67] Santhosh S and Narayana Swamy Ramaiah. (2023). Cloud-based software development lifecycle: A simplified algorithm for cloud service provider evaluation with metric ana. IEEE. 6(2), DOI:10.26599/BDMA.2022.9020016

[68] Catherine De Wolf, Mauro Cordella, Nicholas Dodd, (2023). Whole life cycle environmental impact assessment of buildings: Developing software tool and database support for the EU framework Level(s). Elsevier. 188, Doi:10.1016/j.resconrec.2022.106642

[69] Stefan Walter. (2023). AI impacts on supply chain performance: a manufacturing use case study. Doi:10.1007/s44163-023-00061-9

[70] Victor Vadmand Jensen, Adam Alami (2025). Managing expectations towards AI tools for software development: a multiple-case study. Springer, doi:10.1007/s10257-025-00704-7

[71] Nuruzzaman Faruqui, Priyabrata Thatoi (2024). AI-Analyst: An AI-Assisted SDLC Analysis Framework for Business Cost Optimization. IEEE. 12, DOI:10.1109/ACCESS.2024.3519423

[72] Alcardo Alex Barakabitze (2022). SDN and NFV for QoE-driven multimedia services delivery: The road towards 6G and beyond networks. Elsevier. 214, Doi: 10.1016/j.comnet.2022.109133

[73] Marta Adzgauskaite, Carlos Tam (2025). What helps Agile remote teams to be successful in developing software? Empirical evidence. Elsevier. 177, pp.1-13. Doi: 10.1016/j.infsof.2024.107593

[74] Noshin Tasneem, Hazura Binti Zulzalil (2025). Enhancing Agile Software Development: A Systematic Literature Review of Requirement Prioritization and Reprioritization Techniques. IEEE. 13. DOI:10.1109/ACCESS.2025.3539357

[75] Mohitul Shafir, Partho Protim Saha (2025). The Success Factors of Agile Methodologies in Software Development based on Developing Countries' Software Firms. Elsevier. 256, Doi: 10.1016/j.procs.2025.02.338